markpact

Live Contract Generation

PROMPT: custom

Build a chat API with WebSocket

Model: ollama/qwen2.5-coder:14b

1. Generowanie kontraktu

LLM: ollama/qwen2.5-coder:14b

README.MD

````
# Chat API with WebSocket

This project provides a simple chat application using FastAPI and WebSockets. Users c

## Features
- Real-time messaging via WebSockets
- Connection handling for multiple clients
- Broadcasting messages to all connected clients

## API Endpoints
- **WebSocket /ws** - Establish a WebSocket connection for chat communication.
- **GET /health** - Check the health status of the server.

---

```text markpact:deps python
fastapi
uvicorn[standard]
```

```python markpact:file path=app/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from typing import List

app = FastAPI()

active_connections: List[WebSocket] = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    active_connections.append(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            for connection in active_connections:
                if connection != websocket:
                    await connection.send_text(f"Message from user: {data}")
    except WebSocketDisconnect:
        active_connections.remove(websocket)

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

```bash markpact:run
uvicorn app.main:app --host 0.0.0.0 --port ${MARKPACT_PORT:-8000}
```
````

WALIDACJA + OCZEKIWANIA

Prompt: custom

```
Build a chat API with WebSocket...
```

Model: ollama/qwen2.5-coder:14b
Czas: 61210ms
Rozmiar: 1605 znakow
Linie: 57

WALIDACJA

[V] LLM odpowiedzial
61210ms

[V] README.md zapisany
generated/live/README.md

2. Parsowanie blokow

parse_blocks() -> 4 blokow

README.MD

````
# Chat API with WebSocket

This project provides a simple chat application using FastAPI and WebSockets. Users c

## Features
- Real-time messaging via WebSockets
- Connection handling for multiple clients
- Broadcasting messages to all connected clients

## API Endpoints
- **WebSocket /ws** - Establish a WebSocket connection for chat communication.
- **GET /health** - Check the health status of the server.

---

```text markpact:deps python
fastapi
uvicorn[standard]
```

```python markpact:file path=app/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from typing import List

app = FastAPI()

active_connections: List[WebSocket] = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    active_connections.append(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            for connection in active_connections:
                if connection != websocket:
                    await connection.send_text(f"Message from user: {data}")
    except WebSocketDisconnect:
        active_connections.remove(websocket)

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

```bash markpact:run
uvicorn app.main:app --host 0.0.0.0 --port ${MARKPACT_PORT:-8000}
```
````

WALIDACJA + OCZEKIWANIA

Znaleziono 4 blokow:

```
markpact:deps  (25 chars)
markpact:file=app/main.py  (683 chars)
markpact:run  (65 chars)
markpact:test  (177 chars)
```

WALIDACJA

- [V] markpact:deps
  lang=text, 25 chars
- [V] markpact:file=app/main.py
  lang=python, 683 chars
- [V] markpact:run
  lang=bash, 65 chars
- [V] markpact:test
  lang=text, 177 chars

3. Walidacja blokow

Sprawdzenie wymaganych blokow markpact:*

README.MD

````
# Chat API with WebSocket

This project provides a simple chat application using FastAPI and WebSockets. Users c

## Features
- Real-time messaging via WebSockets
- Connection handling for multiple clients
- Broadcasting messages to all connected clients

## API Endpoints
- **WebSocket /ws** - Establish a WebSocket connection for chat communication.
- **GET /health** - Check the health status of the server.

---

```text markpact:deps python
fastapi
uvicorn[standard]
```

```python markpact:file path=app/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from typing import List

app = FastAPI()

active_connections: List[WebSocket] = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    active_connections.append(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            for connection in active_connections:
                if connection != websocket:
                    await connection.send_text(f"Message from user: {data}")
    except WebSocketDisconnect:
        active_connections.remove(websocket)

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

```bash markpact:run
uvicorn app.main:app --host 0.0.0.0 --port ${MARKPACT_PORT:-8000}
```
````

WALIDACJA + OCZEKIWANIA

Wymagane bloki:

```
markpact:deps  -- zaleznosci
markpact:file  -- kod zrodlowy
markpact:run   -- komenda uruchomienia
```

Opcjonalne:

```
markpact:test  -- testy HTTP
markpact:target -- platforma docelowa
```

Wynik: PASS

WALIDACJA

- [V] markpact:deps obecny
  wymagany
- [V] markpact:file obecny
  wymagany
- [V] markpact:run obecny
  wymagany
- [V] markpact:test obecny
  opcjonalny
- [?] markpact:target
  opcjonalny, brak

4. Analiza zawartosci

Szczegolowa analiza kazdego bloku

README.MD

````
# Chat API with WebSocket

This project provides a simple chat application using FastAPI and WebSockets. Users c

## Features
- Real-time messaging via WebSockets
- Connection handling for multiple clients
- Broadcasting messages to all connected clients

## API Endpoints
- **WebSocket /ws** - Establish a WebSocket connection for chat communication.
- **GET /health** - Check the health status of the server.

---

```text markpact:deps python
fastapi
uvicorn[standard]
```

```python markpact:file path=app/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from typing import List

app = FastAPI()

active_connections: List[WebSocket] = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    active_connections.append(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            for connection in active_connections:
                if connection != websocket:
                    await connection.send_text(f"Message from user: {data}")
    except WebSocketDisconnect:
        active_connections.remove(websocket)

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

```bash markpact:run
uvicorn app.main:app --host 0.0.0.0 --port ${MARKPACT_PORT:-8000}
```
````

WALIDACJA + OCZEKIWANIA

Analiza blokow:

```
Deps: 2 pakietow
  fastapi, uvicorn[standard]
File: app/main.py (23 linii)
Run: uvicorn app.main:app --host 0.0.0.0 --port ${MARKP
Test: 1 assercji HTTP
```

WALIDACJA

- [V] 2 zaleznosci
  fastapi, uvicorn[standard]
- [V] file=app/main.py
  23 lines, sha256=66fec82df48f
- [V] Run command
  uvicorn app.main:app --host 0.0.0.0 --port ${MARKPACT_PORT:-
- [V] 1 testow HTTP
  GET/POST/PUT/DELETE

5. Integralnosc SHA-256

Hash kazdego bloku markpact:*

README.MD

````
# Chat API with WebSocket

This project provides a simple chat application using FastAPI and WebSockets. Users c

## Features
- Real-time messaging via WebSockets
- Connection handling for multiple clients
- Broadcasting messages to all connected clients

## API Endpoints
- **WebSocket /ws** - Establish a WebSocket connection for chat communication.
- **GET /health** - Check the health status of the server.

---

```text markpact:deps python
fastapi
uvicorn[standard]
```

```python markpact:file path=app/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from typing import List

app = FastAPI()

active_connections: List[WebSocket] = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    active_connections.append(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            for connection in active_connections:
                if connection != websocket:
                    await connection.send_text(f"Message from user: {data}")
    except WebSocketDisconnect:
        active_connections.remove(websocket)

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

```bash markpact:run
uvicorn app.main:app --host 0.0.0.0 --port ${MARKPACT_PORT:-8000}
```
````

WALIDACJA + OCZEKIWANIA

Integralnosc danych:

```
README.md: ad6c11d5afaa4c9dbc82a680be9d7cc0

markpact:deps: 0a4bcad277f760e8
markpact:file=app/main.py: 66fec82df48f348d
markpact:run: b49cce15b5712fdd
markpact:test: 44312c5652af6beb
```

Kazdy blok ma unikatowy hash.
Zmiana = nowy hash = wykryta.

WALIDACJA

- [V] README.md hash
  ad6c11d5afaa4c9dbc82a680
- [V] markpact:deps
  sha256=0a4bcad277f760e8
- [V] markpact:file=app/main.py
  sha256=66fec82df48f348d
- [V] markpact:run
  sha256=b49cce15b5712fdd
- [V] markpact:test
  sha256=44312c5652af6beb

Podsumowanie

4	10	0	61.2s
blokow	**passed**	**failed**	**czas**
markpact:*	walidacja	bledy	total

Kroki:

- [V] LLM generate_contract() (61210ms)
  ollama/qwen2.5-coder:14b, 61210ms, 1605 chars
- [V] parse_blocks() (0ms)
  4 blocks
- [V] validate markpact:deps
  present
- [V] validate markpact:file
  present
- [V] validate markpact:run
  present
- [V] Analiza deps
  2 packages: fastapi, uvicorn[standard]
- [V] Analiza file=app/main.py
  23 lines, sha=66fec82df48f
- [V] Analiza run
  uvicorn app.main:app --host 0.0.0.0 --port $MARKPACT_PORT:-8000
- [V] Analiza test
  1 assertions
- [V] SHA-256 README.md
  ad6c11d5afaa4c9dbc82a680

Kontrakt wygenerowany z 1 promptu -> 4 blokow markpact